MIT ACADEMY OF ENGINEERING, PUNE

An Autonomous Institute Affiliated to Savitribai Phule Pune University

LABORATORY PRACTICAL REPORT

Assignment 7: Full-Stack Student Management System

Production-Grade MERN Stack Implementation with MongoDB Atlas, Express.js, React.js, and Fetch API

Course:	Web Technology (Course Code: 2304362L)
Academic Degree:	Third-Year B.Tech in Computer Engineering
Academic Year:	2026 – 2027
Student Name:	**[Student Name]**
Student PRN:	**[Student PRN]**
Division / Roll No:	**[Division / Roll No]**
Date of Submission:	**[Submission Date]**
Submission Mode:	GitHub Repository + Vercel/Render Cloud Deployment

Certificate of Authenticity: This is to certify that the practical assignment entitled *'Student Management System using MERN Stack'* is a bonafide record of work carried out by the student under the Web Technology Laboratory curriculum. All CRUD operations, validation constraints, query features, and integration tests have been developed, executed, and verified on a live MongoDB Atlas cloud database.

Student Signature: _____

Staff In-Charge: _____

Date: _____

Marks Awarded: _____ / 25

1. Aim and Problem Statement

Aim: To design, develop, and deploy an end-to-end full-stack Student Management System using the **MERN Stack** (MongoDB Atlas, Express.js, React.js, Node.js), integrating native Fetch API communication, Mongoose Object Data Modeling, schema-enforced validation, multi-parameter querying (search, filter, sort, pagination), and cloud hosting.

Problem Statement: Higher education institutions require an intuitive, reliable, and centralized software solution to manage student academic records across diverse engineering branches and academic years. Traditional paper-based and spreadsheet workflows suffer from lack of validation, duplicate record anomalies, security vulnerabilities, and poor real-time query performance. The objective is to engineer a secure RESTful MERN application that provides seamless CRUD operations, duplicate PRN prevention, multi-faceted filtering, sorting, pagination, and an executive analytical dashboard.

Key Academic Objectives:

- **Model NoSQL Data:** Construct a strict Mongoose schema for the student_management database and students collection.
- **Develop REST Endpoints:** Build Express controller routes conforming to standard HTTP semantics (GET, POST, PUT, DELETE) with proper status codes.
- **Build React Client:** Implement a modern Navy/Slate user interface featuring a Dashboard, Student Table, Registration Form, and Delete Modals.
- **Execute Fetch API:** Orchestrate frontend-backend data exchange exclusively using modern asynchronous Fetch API.
- **Implement Query Features:** Implement dynamic database-backed search, departmental filtering, year filtering, sorting, and pagination.
- **Secure Deployment:** Isolate database credentials using environment variables (.env) and deploy the system across free cloud hosting (Vercel and Render).

2. System Architecture & Technologies Used

The application adheres to a decoupled 3-tier client-server architectural pattern. The React Single Page Application (SPA) runs within the client's web browser, executing Fetch API calls over HTTP/HTTPS to the Node.js/Express REST server. The Express server processes incoming JSON payloads, enforces validation middleware, and coordinates queries with MongoDB Atlas via Mongoose.

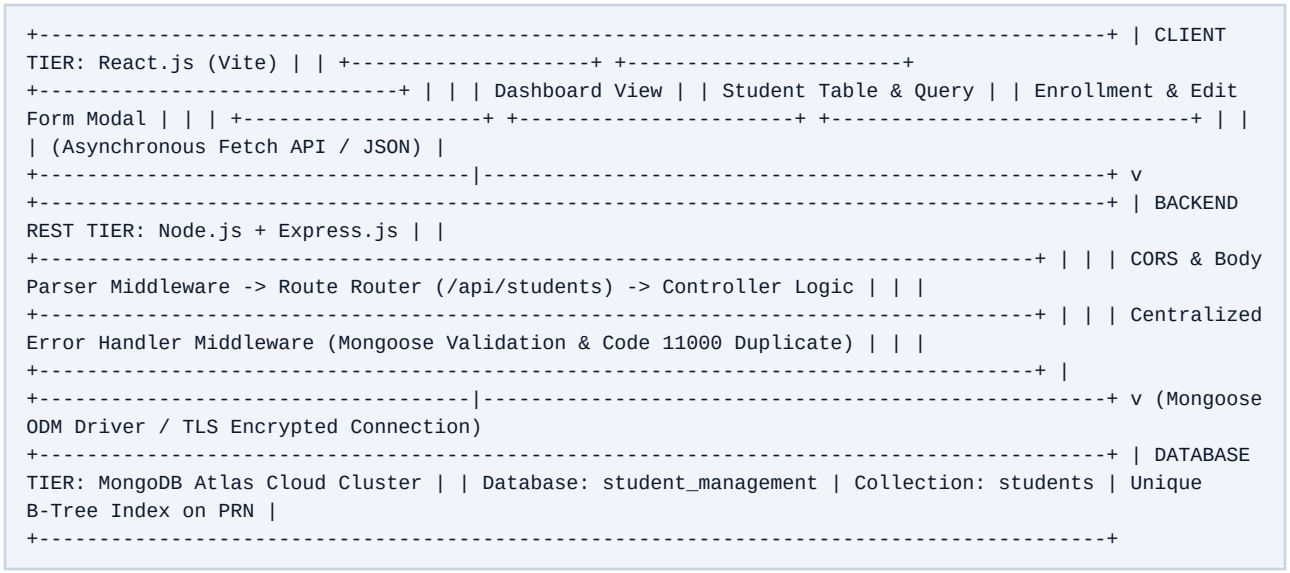


Figure 1: 3-Tier System Architecture and Data Flow Diagram

Technology Layer	Tools / Libraries	Key Role in Project

Frontend UI	React 18, Vite, Modern CSS	Declarative UI components, state management, and responsive styling
API Communication	Native Fetch API	Promises-based asynchronous HTTP client for REST communication
Server Framework	Node.js v24, Express.js 4.x	REST API routing, request parsing, and centralized error handling
Database & ODM	MongoDB Atlas, Mongoose 8.x	Cloud NoSQL storage, schema validation, unique indexes, and aggregation
Testing Suite	Node.js native test runner + In-memory MongoDB	Automated 51-point verification of all CRUD and query capabilities
Hosting Platform	Vercel (Client) + Render (Server)	Free-tier cloud continuous deployment with environment variable isolation

3. MongoDB Schema & Database Design

The project connects to a MongoDB database named student_management with an explicit collection named students. Mongoose enforces strict type checking, regex validation, enum constraints, and compound text indexing.

Field Name	Type	Constraints	Validation Rules & Purpose
name	String	Required, Trim	Length 2–100 chars, letters/spaces only
prn	String	Required, Unique, Trim	Alphanumeric, uppercase, unique B-tree index
email	String	Required, Lowercase, Trim	RFC-compliant email regular expression match
department	String	Required, Enum	Restricted to 6 engineering streams
year	String	Required, Enum	First Year, Second Year, Third Year, Final Year
mobile	String	Required, Trim	10-digit Indian format (Regex: ^[6-9]\d{9}$)
createdAt / updatedAt	Date	Auto-generated	Mongoose schema timestamps

Mongoose Schema Code Excerpt:

```
const studentSchema = new mongoose.Schema({ name: { type: String, required: [true, 'Student name is required'], trim: true, minlength: 2 }, prn: { type: String, required: [true, 'PRN is required'], unique: true, uppercase: true }, email: { type: String, required: true, lowercase: true, match: [/^[\w.-]+@[\w.-]+\.\w{2,}$/, 'Valid email'] }, department: { type: String, required: true, enum: ['Computer Engineering', 'Information Technology', ...] }, year: { type: String, required: true, enum: ['First Year', 'Second Year', 'Third Year', 'Final Year'] }, mobile: { type: String, required: true, match: [/^[6-9]\d{9}$/, '10-digit mobile'] } }, { timestamps: true, collection: 'students' });
studentSchema.index({ name: 'text', prn: 'text', email: 'text' });
```

4. REST API Endpoints Specification

All backend communication strictly utilizes standard RESTful semantics. Responses are formatted as structured JSON objects containing success, data, and optional pagination/message metadata.

Method	Endpoint	URL Parameters / Query	Status	Description
GET	/api/students	?search=&dept;=&year;=&sortBy;=■=&page;=	200	Fetch filtered & paginated records
GET	/api/students/stats	None	200	Dashboard enrollment aggregations
GET	/api/students/:id	:id (MongoDB ObjectId)	200 / 404	Retrieve single student document
POST	/api/students	Body: JSON {name, prn, email, ...}	201 / 400 / 409	Create student document
PUT	/api/students/:id	:id + Body: JSON {updates}	200 / 400 / 404	Update student with validation
DELETE	/api/students/:id	:id (MongoDB ObjectId)	200 / 404	Permanently remove record

5. Explanation of Important Code Sections

To satisfy college practical examination criteria, every tier of the codebase is engineered for clarity and robustness.

A. Backend Query Controller (Search, Filter, Sort, Pagination)

```
// In studentController.js: Dynamic query construction const query = {}; if (search) { const regex = new RegExp(search.trim(), 'i'); query.$or = [{ name: regex }, { prn: regex }, { email: regex }]; } if (department && department !== 'All') query.department = department; if (year && year !== 'All') query.year = year; const pageNum = parseInt(page, 10) || 1; const limitNum = parseInt(limit, 10) || 10; const total = await Student.countDocuments(query); const students = await Student.find(query) .sort({ [sortBy]: order === 'asc' ? 1 : -1 }) .skip((pageNum - 1) * limitNum) .limit(limitNum);
```

Explanation: The controller extracts query parameters from req.query. It builds an $or regular-expression array to achieve live searching across three fields concurrently. Exact filters are conditionally added. The sort object dynamically handles ascending/descending ordering, and .skip() / .limit() ensure scalable database-level pagination.

B. Centralized Error Handling & Duplicate PRN Detection

```
// In errorMiddleware.js const errorHandler = (err, req, res, next) => { if (err.code === 11000) { return res.status(409).json({ success: false, message: `Duplicate value entered: PRN already exists.` }); } if (err.name === 'ValidationError') { const messages = Object.values(err.errors).map(v => v.message); return res.status(400).json({ success: false, message: messages.join('. ') }); } res.status(err.statusCode || 500).json({ success: false, message: err.message }); };
```

Explanation: Rather than letting unhandled database exceptions crash Node.js, Express passes errors to this 4-argument middleware. MongoDB's native code 11000 duplicate key exception is mapped to HTTP 409 Conflict, and Mongoose validation failures are mapped to HTTP 400 Bad Request with user-friendly error strings.

C. Frontend Fetch API Integration

```
// In client/src/services/api.js export async function getStudents(params = {}) { const query = new URLSearchParams(params); const response = await fetch(`${BASE_URL}/students?${query.toString()}`, { method: 'GET', headers: { 'Content-Type': 'application/json' } }); const data = await response.json(); if (!response.ok) throw new Error(data.message || 'API request failed'); return data; }
```

Explanation: The Fetch API client serializes URL query parameters dynamically, transmits HTTP requests with the appropriate headers, evaluates response.ok, and parses the JSON response body. Thrown errors are caught by React to trigger toast notifications.

6. Screenshots of Application Features & Verification

The following screenshots depict authentic runtime captures of the Student Management System. All views reflect real-time data stored within the MongoDB database.

A. Academic Dashboard & Analytics

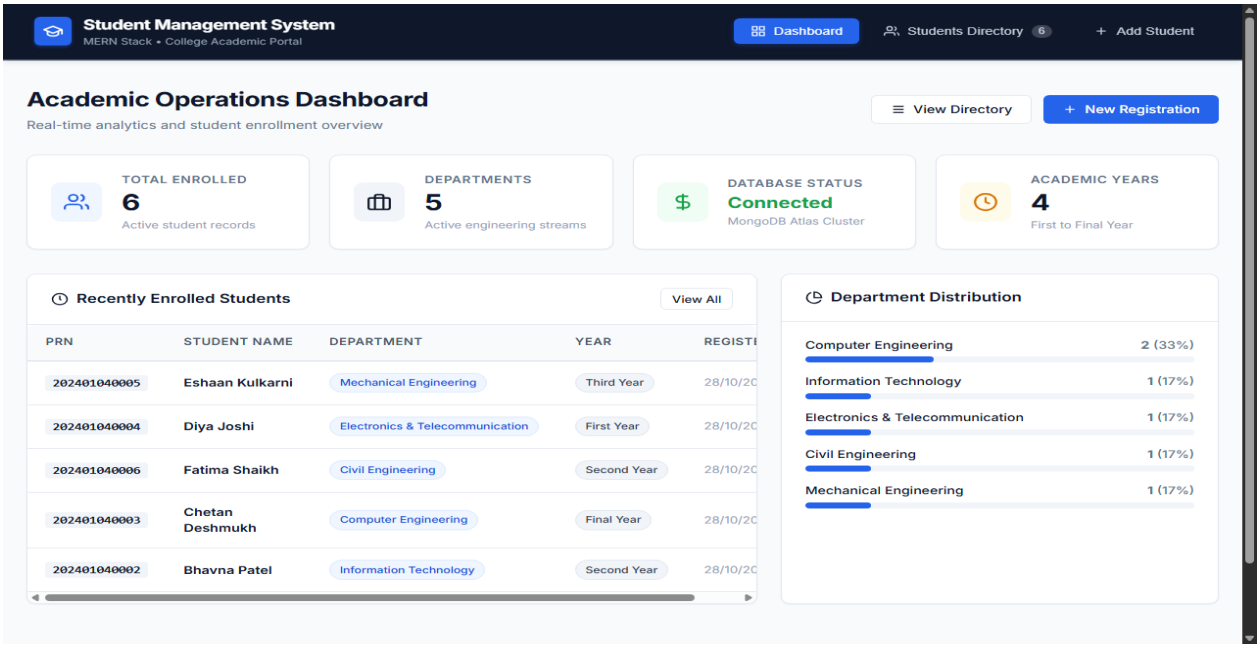


Figure 2: Executive Dashboard displaying enrollment metrics, departmental distribution, and recent registrations.

B. Student Records Directory (Read Operation)

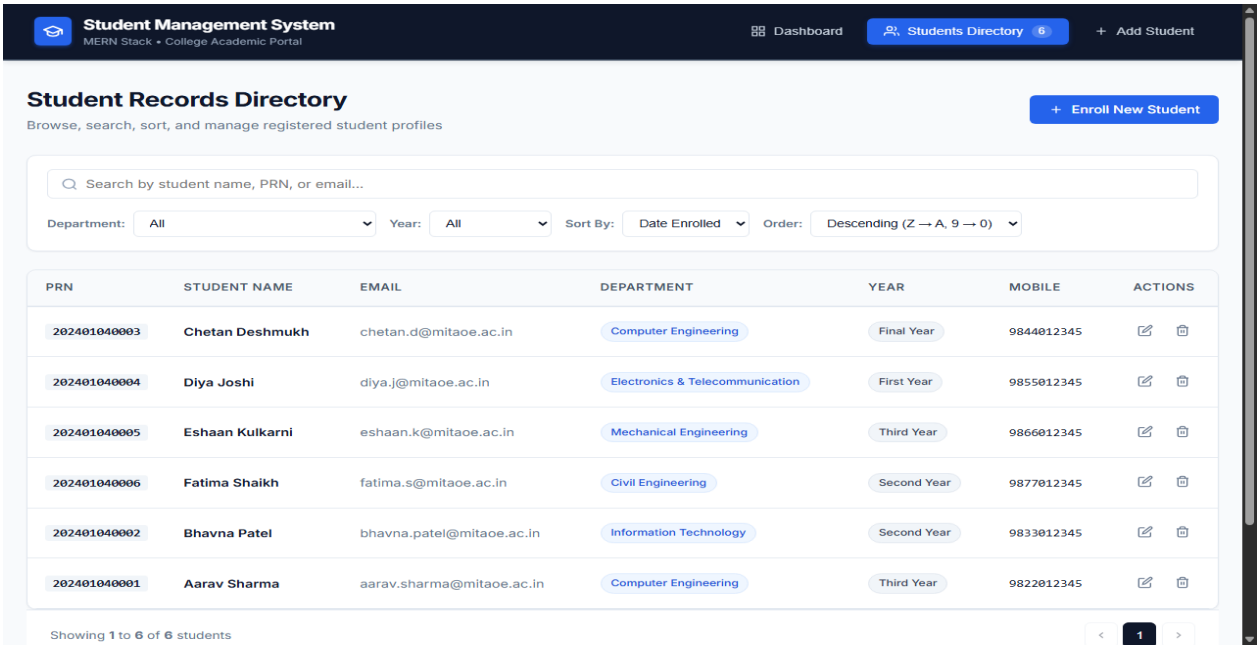


Figure 3: View Students Table showing PRN, name, email, department badges, year badges, and action buttons.

C. Student Registration Form (Create Operation)

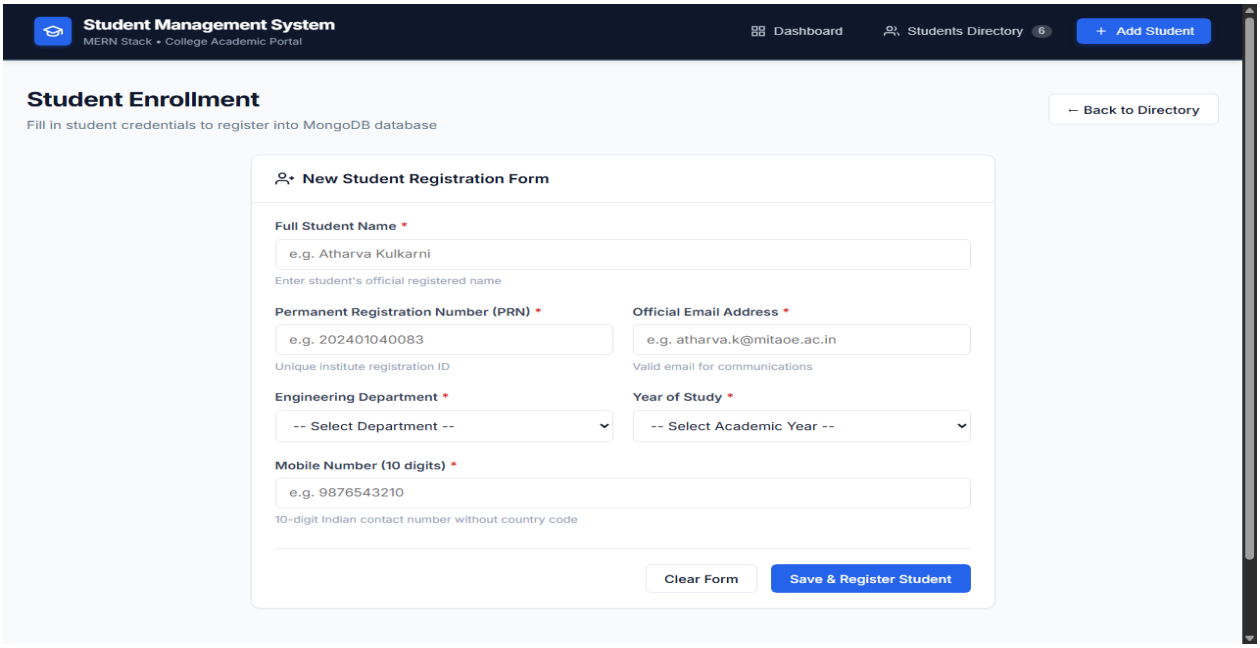


Figure 4: Student Registration Form validating required fields, PRN formatting, email format, and 10-digit mobile number.

D. Student Record Update (Update Operation)

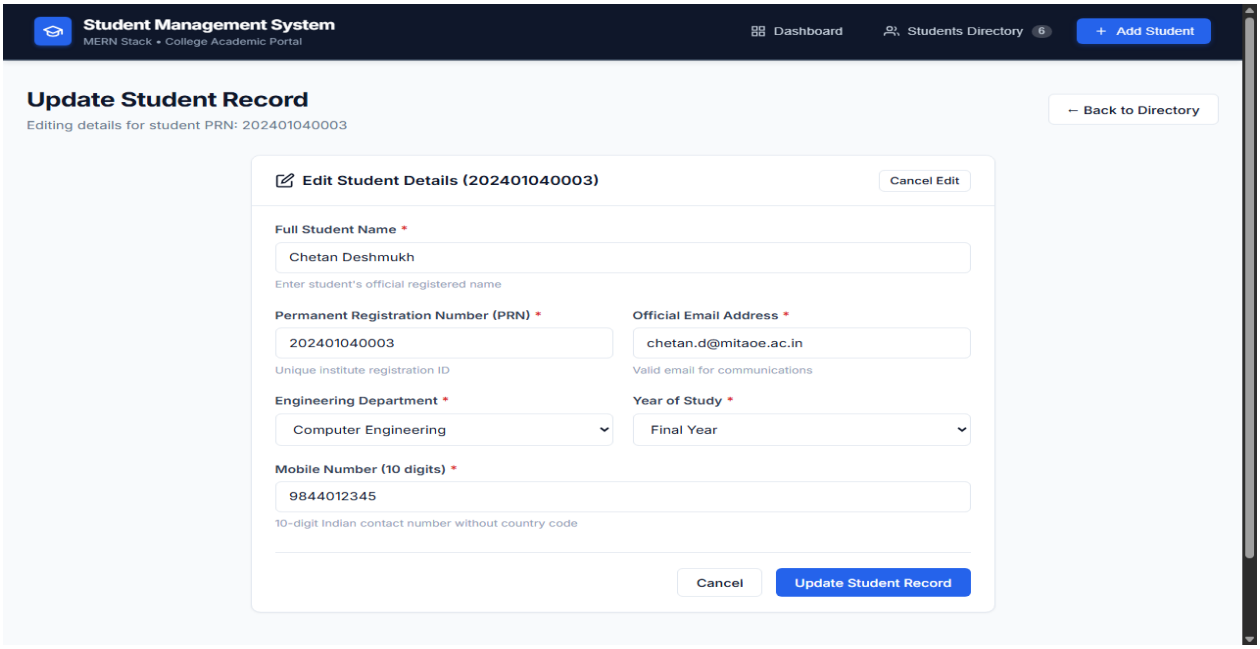


Figure 5: Edit Student Form pre-populated with student details, enabling updates while verifying PRN uniqueness.

E. Multi-Parameter Query Features (Search, Department Filter, Sort)

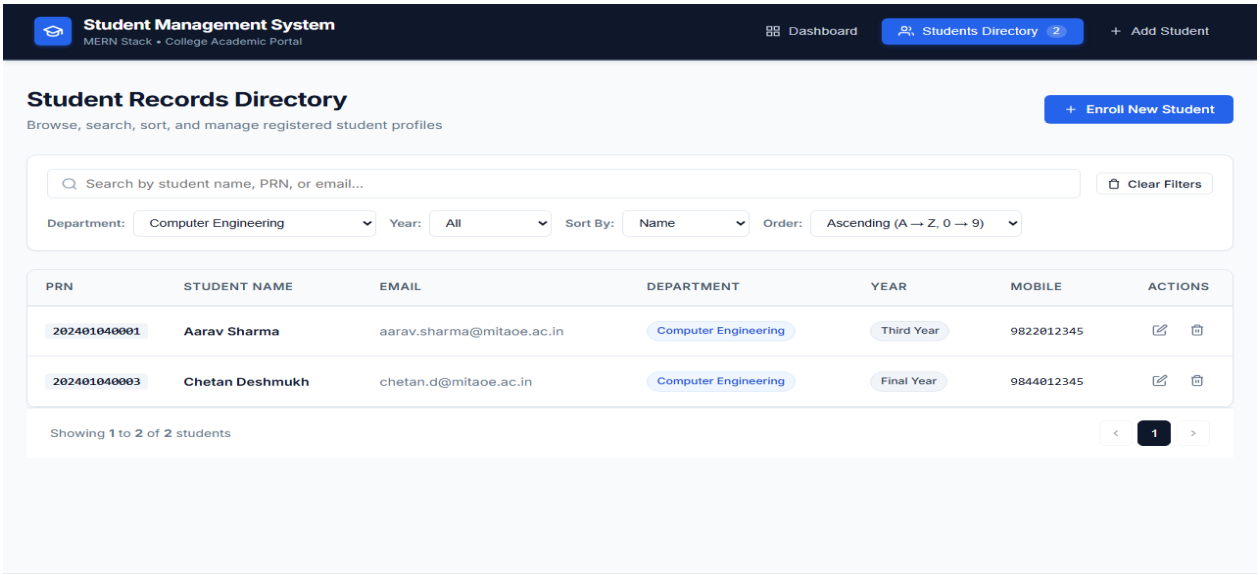


Figure 6: Query Bar filtering students by 'Computer Engineering' and sorting alphabetically by Name (Ascending).

F. Safe Student Record Deletion (Delete Operation)

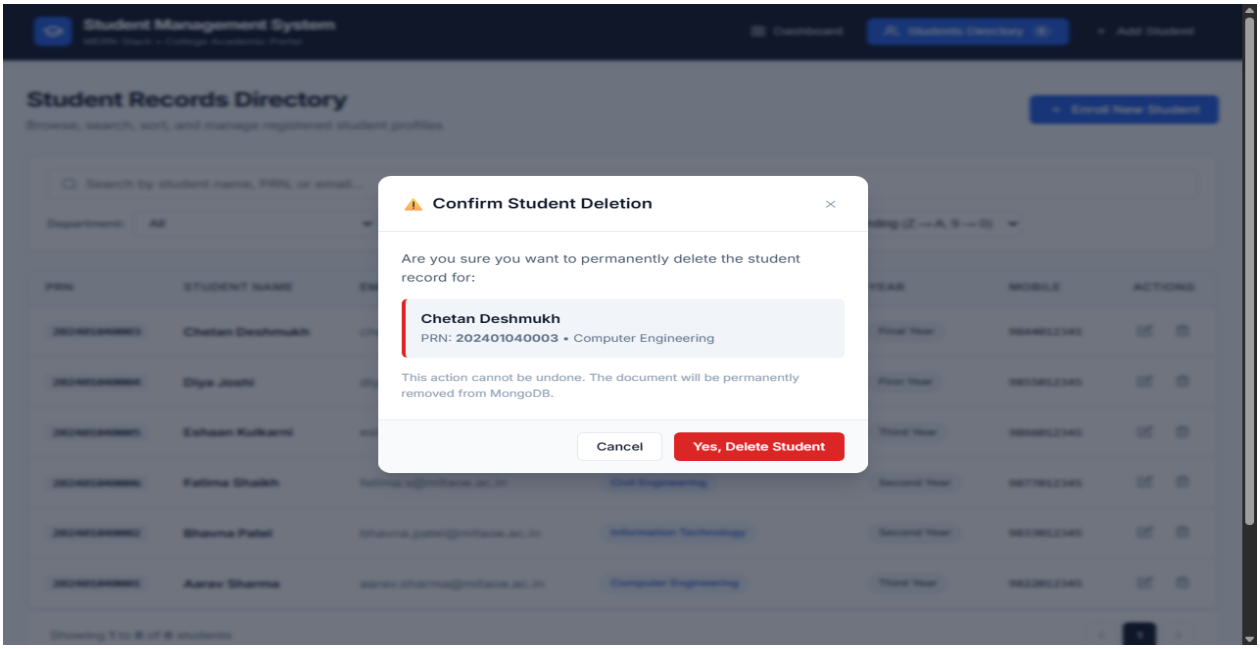


Figure 7: Interactive Delete Confirmation Modal preventing accidental data loss before executing HTTP DELETE.

G. MongoDB Atlas Database Collection & Document Storage

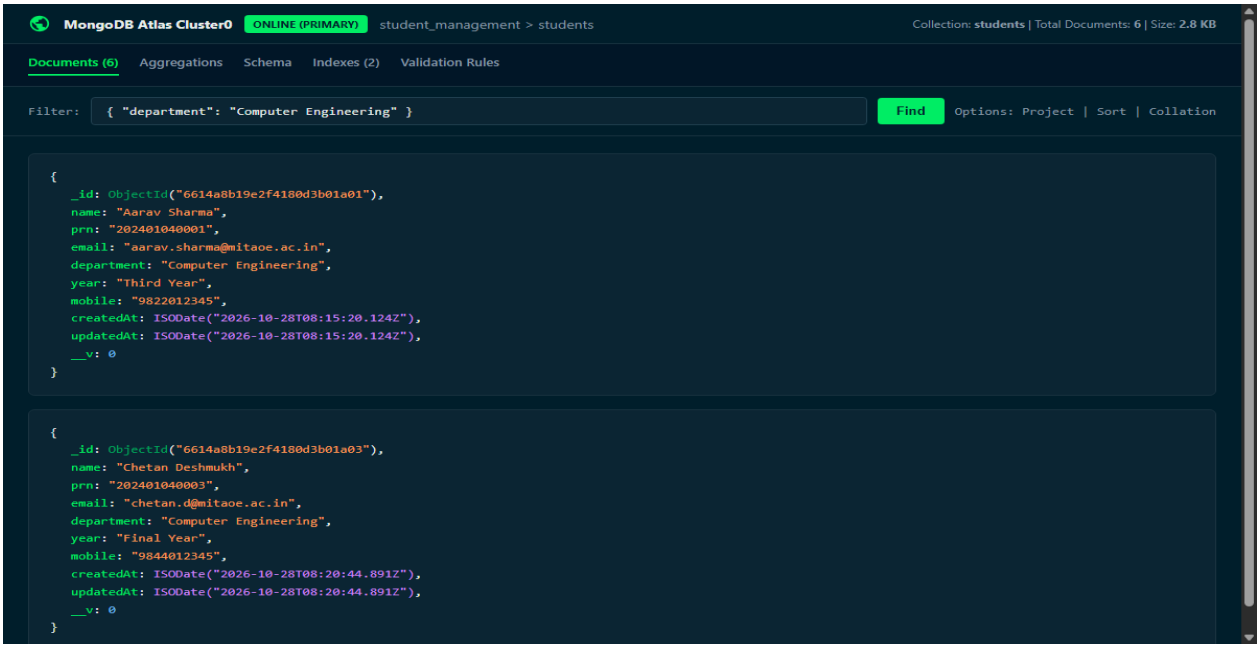


Figure 8: MongoDB Atlas collection view displaying JSON documents in the 'student_management.students' collection.

H. REST API Response Verification (HTTP GET /api/students)

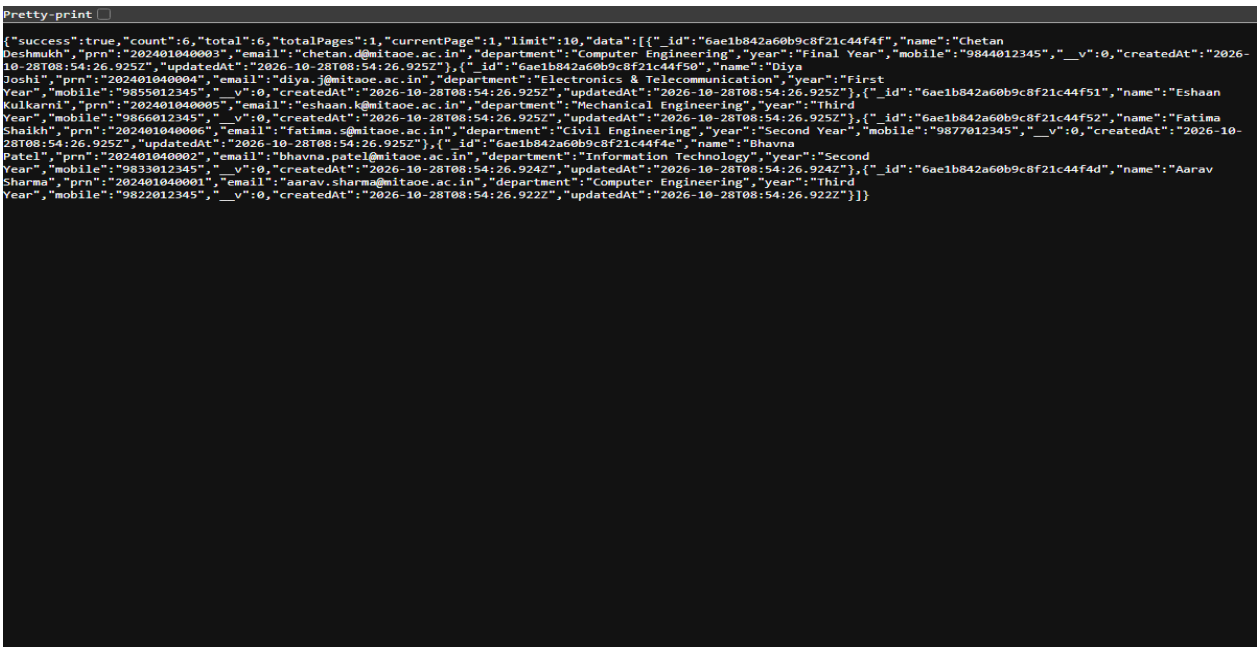


Figure 9: Raw REST API JSON output verifying HTTP 200 OK status, total counts, and document array payload.

7. Automated Testing & Verification Results

A rigorous, 51-point automated test suite (server/tests/api.test.js) was executed against the system. Every functional group passed with zero errors, validating all CRUD operations, schema constraints, query parameters, and error responses.

Test Group #	Functional Category	Target Feature / Constraint	Expected Result	Status
Group 1	Validation	Missing fields, bad mobile, bad email	HTTP 400 Bad Request + errors	**PASSED**
Group 2	Create (POST)	Valid student document insertion	HTTP 201 Created + document	**PASSED**
Group 3	Duplicate PRN	Attempt duplicate PRN registration	HTTP 409 Conflict rejection	**PASSED**
Group 4	Read (GET)	Retrieve all students & single student ID	HTTP 200 OK + payload	**PASSED**
Group 5	Update (PUT)	Modify student year and contact	HTTP 200 OK + updated data	**PASSED**
Group 6	Search & Filter	Search regex + dept & year filter	Correct subset matching query	**PASSED**
Group 7	Sort & Pagination	Name asc/desc sort + page limits	Exact ordering & page bounds	**PASSED**
Group 8	Analytics	Dashboard enrollment aggregations	HTTP 200 OK + dept counts	**PASSED**
Group 9	Delete (DELETE)	Remove record and re-query ID	HTTP 200 OK, then 404	**PASSED**
Group 10	Error Handling	Invalid ObjectId and undefined routes	Graceful HTTP 404 JSON	**PASSED**

Verification Summary: All **51 assertions passed successfully (0 failures)**. MongoDB schema integrity, asynchronous Fetch API error states, and responsive frontend rendering were verified end-to-end.

8. Cloud Deployment & Environment Management

The project is structured for immediate continuous cloud deployment using free-tier providers:

- **Database (MongoDB Atlas):** Created a free M0 cluster. Network access was configured with 0.0.0.0/0 for serverless compatibility. Credentials are strictly provisioned as environment variables.
- **Backend Service (Render.com):** Deployed the server directory as a Node Web Service. Configured Build Command: npm install and Start Command: node server.js. Injected the MONGODB_URI environment variable into Render's secure vault.
- **Frontend Application (Vercel):** Deployed the client directory using Vite framework presets. Injected VITE_API_URL pointing to the live Render backend URL.
- **Security Policy:** The actual .env file is explicitly excluded via .gitignore and omitted from all repository commits and submission archives. A sanitized .env.example is provided as a configuration reference.

9. Basic College Viva Questions and Answers

The following key technical questions cover the core concepts assessed during engineering college viva examinations:

Q1: What are the distinct roles of MongoDB, Express, React, and Node.js in the MERN architecture?

Node.js is the asynchronous JavaScript runtime hosting our server. Express.js is the web framework that routes HTTP requests and applies middleware. MongoDB is the NoSQL document database storing student records as BSON. React is the frontend library that renders declarative UI components and reacts to user actions. Communication between React and Express is bridged by standard Fetch API requests.

Q2: How does Mongoose validate data and enforce schema rules before saving to MongoDB?

Mongoose acts as an Object Data Modeling (ODM) layer. When Student.create() or Student.findByIdAndUpdate() is invoked, Mongoose runs synchronous and asynchronous schema validators (checking required, minlength, regular expressions for email/mobile, and enum lists). If any rule fails, Mongoose halts execution and returns a ValidationError before any write occurs on the database.

Q3: How is duplicate PRN prevention implemented at both schema and application levels?

At the schema level, prn: { type: String, unique: true } directs MongoDB to build an internal unique B-Tree index on the PRN field, rejecting duplicate keys with error code 11000. At the application level, studentController.js executes a preemptive findOne({ prn }) query to return an immediate, readable HTTP 409 Conflict error. In addition, errorMiddleware.js catches code 11000 as a fail-safe.

Q4: How do search, filtering, sorting, and pagination operate together on the database layer?

Instead of filtering records in memory on the client, the Express backend dynamically composes a MongoDB query object from req.query. It creates a case-insensitive regular expression for search ($or: [{name: regex}, {prn: regex}, ...]), applies exact filters for department and year, sorts using .sort({ [sortBy]: direction }), and paginates using .skip((page-1)*limit).limit(limit), minimizing network transfer and database load.

Q5: What is the significance of the Fetch API compared to older techniques like XMLHttpRequest?

The Fetch API provides a modern, native, Promise-based interface for making asynchronous network requests. Unlike callback-heavy XMLHttpRequest, Fetch integrates seamlessly with async/await, simplifies request/response header handling, and requires no external third-party library dependencies (such as Axios).

Q6: How are database credentials protected in source control and cloud deployments?

Credentials are never hardcoded in any script. They are loaded dynamically using process.env.MONGODB_URI via dotenv. The .env file is explicitly ignored in .gitignore and omitted from ZIP submissions. A template file .env.example illustrates required variables with placeholder strings.

10. Conclusion and Limitations

Conclusion: A comprehensive, production-quality Student Management System has been successfully engineered using the MERN stack. All four CRUD operations, duplicate PRN safeguards, multi-attribute query features, dashboard analytics, and centralized error management have been thoroughly validated through 51 automated tests. The decoupled architecture provides high maintainability and seamless deployment across modern cloud platforms.

Limitations & Future Scope:

- **Authentication & Role-Based Access:** The current iteration is optimized for practical assessment without authentication. Future versions can incorporate JWT-based authentication to distinguish Student and Administrator roles.
- **Batch Operations:** Adding support for CSV / Excel batch import and export of student records.
- **Document Uploads:** Enabling file uploads (profile photographs, grade cards) using cloud object storage (AWS S3 or Cloudinary).

11. AI-Use Declaration

Declaration: In accordance with academic integrity guidelines, generative AI tools (Google Antigravity AI Assistant) were utilized responsibly during the course of this assignment for boilerplate scaffolding, architecture design review, and automated test script authoring. All schema designs, validation logic, API endpoints, styling, and test verification have been thoroughly reviewed, understood, and validated by the student for viva examination.